Robotics Programming Challenge 1

Forward and inverse position kinematics of the Franka Emika Panda · MCTR911, Winter 2026

Name / ID: _____

This report documents the DH model, the forward kinematics (FK), the inverse kinematics (IK) and how each was verified. The code is in `panda_kinematics/kinematics.py`; the verification script is `test_kinematics.py`. The robot model is `panda.xml` from Google DeepMind's MuJoCo Menagerie (commit 71f066a, as given in the handout). FK, IK and joint limits use the seven arm joints only; the two finger joints are fixed by the gripper command.

1. DH model (standard convention)

Each link uses the standard DH transformation of Tutorial 03 (slide 13): a rotation θ_i about z_{i-1}, a translation d_i along z_{i-1}, a translation a_i along x_i and a twist α_i about x_i. The joint variable is $\theta_i = q_i$; α, a and d are fixed geometry read once from panda.xml.

$$ {}^{i-1}T_i = \mathrm{Rot}_z(\theta_i) \cdot \mathrm{Trans}_z(d_i) \cdot \mathrm{Trans}_x(a_i) \cdot \mathrm{Rot}_x(\alpha_i) $$

Table 1 – DH parameters and their source in panda.xml

Link i	θ_i	d_i (m)	a_i (m)	α_i	Source in panda.xml
1	q_1	0.333	0	−π/2	d: link1 pos (0, 0, 0.333); α: link2 quat (1, −1, 0, 0) = −90° about x
2	q_2	0	0	+π/2	α: link3 quat (1, 1, 0, 0) = +90° about x
3	q_3	0.316	0.0825	+π/2	d: link3 pos (0, −0.316, 0); a, α: link4 pos (0.0825, 0, 0), quat (1, 1, 0, 0)
4	q_4	0	−0.0825	−π/2	a, α: link5 pos (−0.0825, 0.384, 0), quat (1, −1, 0, 0)
5	q_5	0.384	0	+π/2	d: link5 pos y = 0.384; α: link6 quat (1, 1, 0, 0)
6	q_6	0	0.088	+π/2	a, α: link7 pos (0.088, 0, 0), quat (1, 1, 0, 0)
7	q_7	0	0	0	last joint; frame 7 coincides with the MuJoCo link7 frame

Notes on the table:

- Consecutive Panda axes alternate between twist joints (1, 3, 5, 7) and bend joints (2, 4, 6), so consecutive axes are perpendicular and every α is ±90° (sign from the quaternion).
- panda.xml is written in modified (Craig) DH, where frame i sits on joint i. In standard DH frame i sits on joint i+1, so each a and α appears one row earlier than the body it is read from.
- Panda has parallel axes, so other consistent DH tables exist; this one was chosen so that frame 7 equals the simulator's link7 frame, which makes the hand transform a single fixed matrix.

Fixed transform link 7 → hand

From `<body name="hand" pos="0 0 0.107" quat="0.9238795 0 0 −0.3826834">`: 0.107 m along z_7 and a rotation of −45° about z (cos 22.5° = 0.924, −sin 22.5° = −0.383).

$$ T_{7,\mathrm{hand}} = \begin{vmatrix} 0.7071 & 0.7071 & 0 & 0 \\ -0.7071 & 0.7071 & 0 & 0 \\ 0 & 0 & 1 & 0.107 \\ 0 & 0 & 0 & 1 \end{vmatrix} $$

Table 2 – Joint limits (rad), from the joint range attributes (default ±2.8973)

	q_1	q_2	q_3	q_4	q_5	q_6	q_7
Q_MIN	−2.8973	−1.7628	−2.8973	−3.0718	−2.8973	−0.0175	−2.8973
Q_MAX	2.8973	1.7628	2.8973	−0.0698	2.8973	3.7525	2.8973

2. Forward kinematics

$$ {}^{0}T_{hand}(q) = {}^{0}T_{1}(q_1) \cdot {}^{1}T_{2}(q_2) \cdot \ldots \cdot {}^{6}T_{7}(q_7) \cdot T_{7,hand} $$

`forward_kinematics(q)` starts from the identity, multiplies the seven DH matrices from the base outwards (each built by `dh_transform(α, a, d, q_i)`), then multiplies $T_{7,hand}$. It returns the 4×4 pose of the hand in link0: the last column is the position, the 3×3 block the orientation. The order of multiplication follows the kinematic chain because each link is described relative to the previous one.

Table 3 – Hand check at q = 0 (all links stacked vertically, hand pointing down)

Coordinate	Hand calculation	FK result
z	0.333 + 0.316 + 0.384 − 0.107	0.926 m
x	+0.0825 − 0.0825 + 0.088	0.088 m
y	—	0
hand z-axis	points down	(0, 0, −1)

q = 0 violates the joint 4 limit, so it is only an analytical check, not a commanded pose.

Table 4 – FK verification against MuJoCo (test_kinematics.py)

Test	Result
2,000 random q inside the limits: hand position, FK vs MuJoCo body 'hand'	max error 1.0×10^{-12} mm
Same, hand orientation	max error 2×10^{-6} °
DH frame 7 vs MuJoCo link7 frame	max $\lvert\Delta T\rvert = 3.8 \times 10^{-15}$
Joint limits vs panda.xml ranges	identical
Geometric Jacobian vs finite differences of FK	max difference 4.2×10^{-7}

3. Inverse kinematics

Method: Newton–Raphson (Tutorial 04), $q_{n+1} = q_n - (\partial F/\partial q)^{-1} F(q_n)$ with $F(q) = FK(q) - target$. The Panda has 7 joints for a 6-D pose, so the 6×7 Jacobian J has no inverse; it is replaced by the damped least-squares (Levenberg–Marquardt) pseudo-inverse:

$$ e = [\ p_{target} - p\ ;\ \log(R_{target}\ R^{T})\ ] \quad \text{(6×1 pose error in link0)} $$

$$ \Delta q = J^{T}\ (J\ J^{T} + \lambda^{2}\ I)^{-1}\ e, \quad \lambda^{2} = \mu\ \|e\|^{2} + \lambda_{bias} $$

J is the geometric Jacobian built from the FK frames (column i: $z_{i-1} \times (p_{hand} - o_{i-1})$ and z_{i-1}). With $\lambda = 0$ the update is exactly Newton–Raphson with the pseudo-inverse $J^{T}(JJ^{T})^{-1}$, i.e. the minimum-norm joint step.

Algorithm (one attempt):

- Clamp the seed q_0 (current joint state) into the limits; compute e.
- Stop with success when $\|e_{pos}\| < 0.1$ mm and $\|e_{rot}\| < 0.001$ rad.
- Compute Δq. Joints already on a limit and pushed outward are frozen (column of J set to 0) and Δq is recomputed.
- Cap the step so that no joint moves more than 0.2 rad; clamp q + Δq into the limits.
- Accept the step only if ‖e‖ decreases (then μ ← μ/2); otherwise reject it and set μ ← 4μ.
- Repeat (at most 150 iterations).

If the attempt from q_0 fails, up to 10 deterministic restart seeds (mid-range and fixed random seeds) are tried, and the converged solution closest to q_0 is returned. A 0.2 s time budget keeps the 20 Hz node responsive.

Table 5 – How the four required problems are handled

Problem	Handling
Multiple solutions	Seeded with the current joint state and minimum-norm steps, so the solution nearest the current configuration is returned; restarts keep the converged solution closest to q_0. In closed loop the commanded q changed by at most 0.0055 rad per tick (no branch switching).
Joint limits	Enforced inside the loop (clamp after every step, active-set freezing of joints on a limit). Every returned q is within Q_MIN / Q_MAX.
Singularities	Damping λ^2 keeps the step finite when a singular value of J tends to 0 (gain $\sigma/(\sigma^2+\lambda^2)$ instead of $1/\sigma$); λ_{bias} keeps the matrix invertible at exact rank loss; the 0.2 rad step cap bounds every move. At $q_5 = 0$ the axes of joints 4 and 6 are parallel (0.39 m apart) but the 7-DoF Jacobian keeps full rank ($\sigma_{min} \approx 0.17$); near-rank loss occurs e.g. at $q_2 = 0$ with the elbow stretched ($\sigma_{min} \approx 0.003$–$0.03$). All such cases were solved without failure.
Unreachable targets	Invalid poses (not 4×4, NaN, non-orthonormal R) are rejected; targets farther than 0.858 m from the shoulder (sum of rigid segment lengths) are rejected immediately; otherwise failure of all attempts sets the flag to False. On failure the function returns (q_0, False), so the arm holds its position.

Table 6 – IK verification (every solution checked with MuJoCo's own kinematics)

Test set	Success	Max error	Median time
300 random targets, seed near the target	300 / 300	0.10 mm, 0.04°	0.9 ms
300 random targets, seed = home pose	299 / 300	0.10 mm, 0.02°	3.6 ms
300 random targets, seed = all zeros	299 / 300	0.10 mm, 0.03°	10 ms
Targets with $q_5 = 0$, $q_6 = 0$, $q_2 = 0$ + stretched elbow, near-singular seed (4 × 100)	400 / 400	0.10 mm, 0.05°	0.9–71 ms
Unreachable / invalid targets (6 cases)	all flagged False, q_0 held	—	≤ 0.2 s

The two far-seed failures used up the 0.2 s budget before a restart converged; they are reported as False, never as a wrong pose.

4. Target pose and simulation result

The target was generated as the handout suggests: a configuration inside the limits, Q_TARGET = (0.4, 0.2, −0.3, −2.0, 0.3, 2.2, 0.3) rad, passed through FK. In MuJoCo this configuration has no contacts and the hand is 0.37 m above the floor.

$$\text{TARGET_POSE} = \begin{vmatrix} 0.9145 & 0.4046 & 0.0104 & 0.5867 \\ 0.3959 & -0.8997 & 0.1837 & 0.0745 \\ 0.0837 & -0.1638 & -0.9829 & 0.3658 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Running the node with the mujoco_ros2 bridge in POSITION mode, the arm moved from the initial configuration to the target and held it. From the measured /joint_state, FK gives the hand at (0.586, 0.074, 0.354) m, 11.7 mm and 1.3° from the target. The IK command itself is exact (FK of the commanded q equals the target to 0.1 mm); the residual is a steady-state error of the simulator's position controller under gravity, mostly at joint 2 (measured 0.207 rad vs 0.191 rad commanded).

5. Files and how to run

File	Content
panda_kinematics/kinematics.py	DH_PARAMS, T_L7_HAND, Q_MIN, Q_MAX, Q_TARGET / TARGET_POSE, dh_transform, forward_kinematics, inverse_kinematics, ROS 2 node
setup.py, setup.cfg, package.xml	ament_python package; entry point kinematics = panda_kinematics.kinematics:main
test_kinematics.py	FK / Jacobian / IK verification against MuJoCo (no ROS needed)

```
colcon build --packages-select panda_kinematics && source install/setup.bash
ros2 launch mujoco_ros2 example.py # simulator, control_mode POSITION
ros2 run panda_kinematics kinematics # this node
python3 test_kinematics.py <path>/franka_emika_panda/panda.xml # verification
```